

QUESTÃO 3 (3,0 pts) — MODELAGEM COM DATA (IMÓVEL E BAIRRO)

ENUNCIADO: (a) Defina Imóvel com Casa e Apartamento (metragem quadrada e preço). (b) Defina Bairro com Pompeia e SantaMaria (lista de imóveis [Imóvel], descrição e preço médio m²).

```
-- SOLUÇÃO (a): metragem (Float) e preço (Float)
data Imóvel = Casa Float Float
            | Apartamento Float Float
            deriving (Show, Eq)

-- SOLUÇÃO (b): lista [Imóvel], descrição (String), preço médio m² (Float)
data Bairro = Pompeia [Imóvel] String Float
            | SantaMaria [Imóvel] String Float
            deriving (Show, Eq)

-- Alternativa Record Syntax (nomes explícitos nos campos):
data Imóvel = Casa { metragem :: Float, preco :: Float } | Apartamento { metragem :: Float, preco :: Float } deriving (Show, Eq)
data Bairro = Pompeia { moveis :: [Imóvel], desc :: String, precoMedio :: Float } | SantaMaria { moveis :: [Imóvel], desc :: String, precoMedio :: Float } deriving (Show, Eq)
```

QUESTÃO 1 (1,0 pt) — LISTA PRÓPRIA (:>:)

ENUNCIADO: Dado List a = a :>: (List a) | Nulo, implemente removerElementoInicio.

```
data List a = a :>: (List a) | Nulo deriving (Show, Eq)

-- Resposta oficial (remover primeiro):
removerElementoInicio :: List a → List a
removerElementoInicio (x :>: xs) = xs
removerElementoInicio Nulo       = Nulo

-- 🚩 DICA DO PROFESSOR: REMOVER DO MEIO (por valor):
removerMeio :: (Eq a) ⇒ a → List a → List a
removerMeio _ Nulo = Nulo
removerMeio val (x :>: xs)
  | val == x = xs
  | otherwise = x :>: removerMeio val xs -- mantém e segue

-- 🚩 REMOVER POR POSIÇÃO (índice 0, 1...):
removerPos :: Int → List a → List a
removerPos _ Nulo = Nulo
removerPos 0 (_ :>: xs) = xs
removerPos n (x :>: xs) = x :>: removerPos (n-1) xs
```

QUESTÃO 2 (2,0 pts) — ÁRVORES BINÁRIAS

ENUNCIADO: Raiz 30; Esq: Galho 20 (Nulo, Folha 3); Dir: Galho 45 (Folha 41, Nulo). Escreva a expressão.

```
data Arvore a = Galho a (Arvore a) (Arvore a)
              | Folha a | Nulo deriving (Show, Eq)

-- Expressão oficial da questão:
arvoreResposta =
  Galho 30 (Galho 20 Nulo (Folha 3))
          (Galho 45 (Folha 41) Nulo)

-- Funções úteis de árvore (caso peça):
altura Nulo = 0
altura (Folha _) = 1
altura (Galho _ e d) = 1 + max (altura e) (altura d)

somaArv Nulo = 0
somaArv (Folha x) = x
somaArv (Galho x e d) = x + somaArv e + somaArv d
```

SUAS ANOTAÇÕES: DATA, TYPE E ESTRUTURAS RECURSIVAS (SUAS PALAVRAS)

- data** (Caixinhas com Etiquetas): Inventa um tipo novo que não existia no Haskell. O pipe | é simplesmente um OU (uma caixinha com etiqueta Casa OU Apartamento). Os construtores SEMPRE começam com letra MAIÚSCULA!
- type** (Apelido): Não cria tipo novo, só dá apelido pra tuplas ou listas que já existem: type Nome = String | type Ponto = (Float, Float) | type Coord = (Int, Int)
- Estruturas Recursivas (Boneca Russa): caixinha dentro de caixinha. :>: é o cons (junta cabeça com cauda) e Nulo é a lista vazia []. Para remover o primeiro elemento, pegamos a cauda xs e jogamos fora a cabeça x!

QUESTÃO 4 (2,0 pts) — FUNÇÃO BAR (FILTER) E TESTE DE MESA DO "FATEC"

ENUNCIADO: `bar func (x:xs) | func x == True = x : bar func xs | otherwise = bar func xs.`
(a) O que essa função faz? (b) Faça o teste de mesa com `bar (\x -> elem x "FEfe") "FATEC"`

-- (a) Resposta: A função 'bar' filtra a lista com base no predicado 'func'.
Se 'func x == True', o elemento 'x' é mantido; caso contrário, é descartado. Trata-se da reimplementação de 'filter'.

Passo	Chamada	x	xs	Condição elem x "FEfe"	Ação / Retorno Parcial
1	bar func "FATEC"	'F'	"ATEC"	True ('F' está em "FEfe")	'F' : bar func "ATEC" (Mantém 'F')
2	bar func "ATEC"	'A'	"TEC"	False ('A' não está)	bar func "TEC" (Joga fora 'A')
3	bar func "TEC"	'T'	"EC"	False ('T' não está)	bar func "EC" (Joga fora 'T')
4	bar func "EC"	'E'	"C"	True ('E' está em "FEfe")	'E' : bar func "C" (Mantém 'E')
5	bar func "C"	'C'	""	False ('C' não está)	bar func "" (Joga fora 'C')
6	bar func ""	-	-	Caso base ([] vazia)	[]

-- Retorno final reconstruído: 'F' : 'E' : [] = ['F', 'E'] = "FE"

QUESTÃO 5 (1,0 pt) — AVALIAÇÕES RÁPIDAS E TIPOS (:T)

- (a) `:t filter odd` $\Rightarrow$ **Integral a $\Rightarrow$ [a] $\rightarrow$ [a]** (odd requer inteiros)
- (b) `map (\x  $\rightarrow$  x * 2) [1,3,5,7]` $\Rightarrow$ **[2, 6, 10, 14]** (multiplica cada elemento da lista por 2)
- (c) `:t ("abc", True)` $\Rightarrow$ **[(Char), Bool]** ou (String, Bool)
- (d) `foldl (+) 0 [2,4,6,8]` $\Rightarrow$ **20** (tritador/acumulador: 0 + 2 + 4 + 6 + 8 = 20)
- (e) `[x * 3 | x  $\leftarrow$  [1..6], x > 3]` $\Rightarrow$ **[12, 15, 18]** (filtro x > 3 gera 4, 5, 6; vezes 3 dá 12, 15, 18)

SUAS ANALOGIAS: A LINHA DE FÁBRICA (MAP, FILTER, FOLDL)

- map** (0 Transformador): passa cada item da esteira e aplica uma ferramenta:
`map (\x  $\rightarrow$  x * 2) [1, 3]  $\Rightarrow$  [2, 6]` (o tamanho da lista nunca muda, só o conteúdo é transformado)
- filter** (0 Porteiro): olha cada item: se der True passa, se der False vai pro lixo:
`filter odd [1, 2, 3, 4]  $\Rightarrow$  [1, 3]`
- foldl** (0 Triturador / Acumulador): pega todos os elementos da lista e amassa em um único valor final:
`foldl (-) 10 [1, 2, 3]  $\Rightarrow$  ((10 - 1) - 2) - 3 = 4` (acumula da esquerda para direita)
`foldr (-) 10 [1, 2, 3]  $\Rightarrow$  1 - (2 - (3 - 10)) = -8` (acumula da direita para esquerda)